SENG8323: SOFTWARE MODELING DESIGN

LECTURER: Mr. Placide RWEGO MUCYO

CONTACTS: Phone number: 0788496945

E-mail: mupla2@gmail.com

Academic Year 2025-2026

INTRODUCTION

- Software Modeling and Design help developers plan, visualize, and implement systems effectively. By using Unified Modeling Language (UML), design patterns, and architectural principles, software engineers can create scalable, maintainable, and high-quality applications.
- Models are used during the requirements engineering process to help derive the requirements for a system, during the design process to describe the system to engineers implementing the system and after implementation to document the system's structure and operation

In model-based software design and development, **software modeling** is used as an essential part of the software development process. Models are built and analyzed prior to the implementation of the system, and are used to direct the subsequent implementation.

A better understanding of a system can be obtained by considering it from different perspectives (also referred to as multiple views) such as requirements models, static models, and dynamic models of the software system. A graphical modeling language such as UML helps in developing, understanding, and communicating the different views.

INTRODUCTION

- **GRADING**
- Assignment, Quizzes, Participation, Projects : 30%
- Mid Semester: 30%
- Final Exam: 40%
- **TOTAL: 100%**

Course Content

- **Part I Introduction to Software engineering**
- Introduction to Software Modeling and Design and the UML (Unified Modeling Language)
- Software architectural design principles
- Overview of Software modeling and design method

Part II Software Modeling

- Model software behaviors and **entities** through use case model
- Associations(**relationships**) in use case modeling
- Static and dynamic modeling principles and practices
- Dynamic modeling principles and practices

Part III Architectural Design

- Overview of architectural design decisions
- Architectural views
- Application architectures
- Design client/server software architecture
- Service-oriented architecture
- Component- based software architecture
- Real-time software architecture

Overview of SE

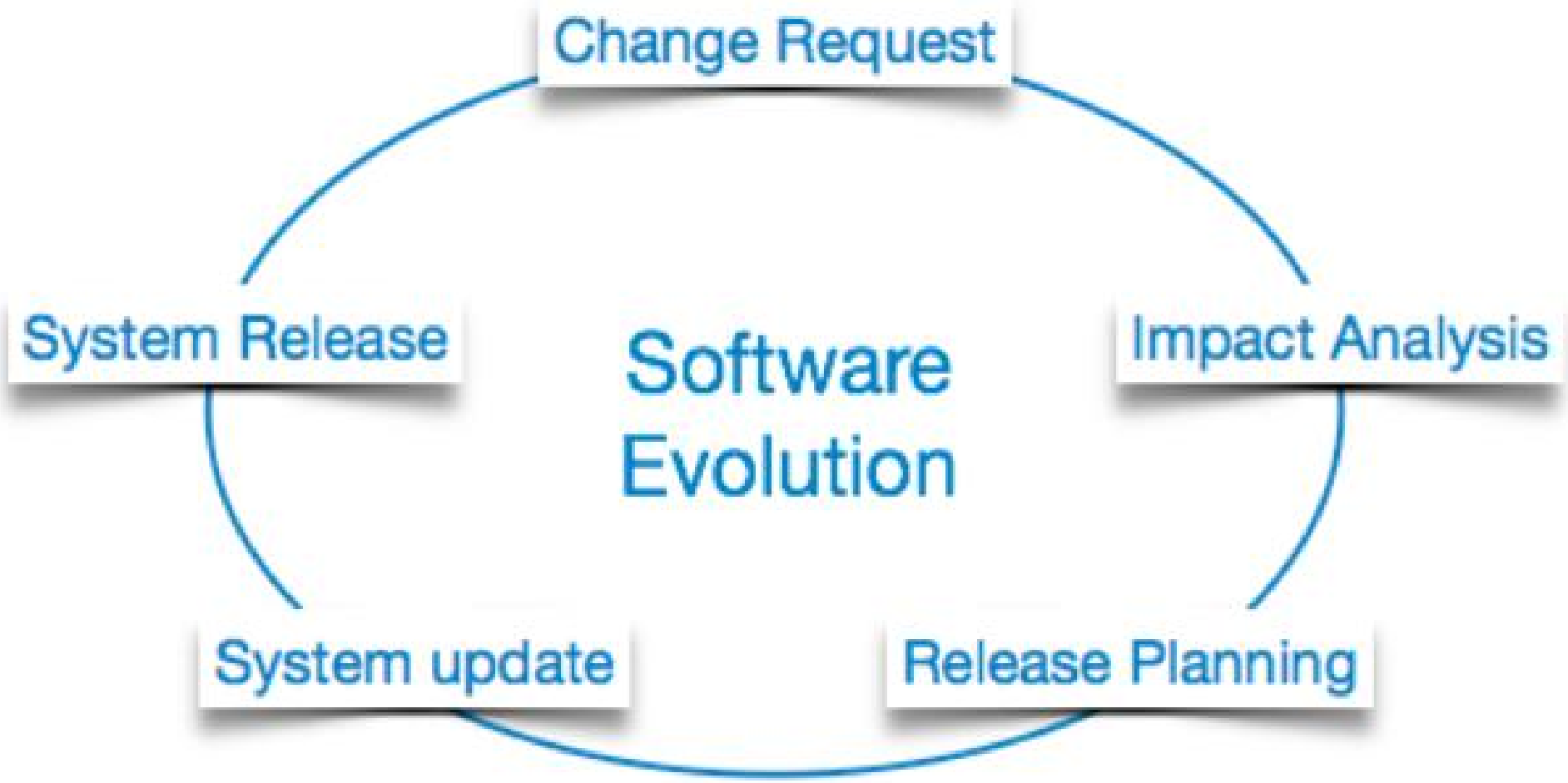
- The term is made of two words, software and engineering.
- Software is more than just a program code.
- A program is an executable code, which serves some computational purpose.
- Software is considered to be collection of executable programming code, associated libraries and documentations.
- Software, when made for a specific requirement is called software product.
- **Engineering** on the other hand, is all about developing products, using well-defined, scientific principles and methods.

Software Product
Requirements
System Analysis
System Design
Code Design
Testing
Deployment
Maintenance
Updates

- Software engineering is an engineering branch associated with development of software product using well-defined scientific principles, methods and procedures.
- The outcome of software engineering is an efficient and reliable software product.

Software Evolution

- The process of developing a software product using software engineering principles and methods is referred to as **software evolution.**
- This includes the initial development of software and its maintenance and updates, till desired software product is developed, which satisfies the expected requirements


Change Request
Impact Analysis
Release Planning
System update
System Release
Software Evolution

- Evolution starts from the requirement gathering process.
- After which developers create a prototype of the intended software and show it to the users to get their feedback at the early stage of software product development.
- The users suggest changes, on which several consecutive updates and maintenance keep on changing too.
- This process changes to the original software, till the desired software is accomplished.
- Even after the user has desired software in hand, the advancing technology and the changing requirements force the software product to change accordingly.
- Re-creating software from scratch and to go one-on-one with requirement is not feasible.
- The only feasible and economical solution is to update the existing software so that it matches the latest requirements.

Software Evolution Laws

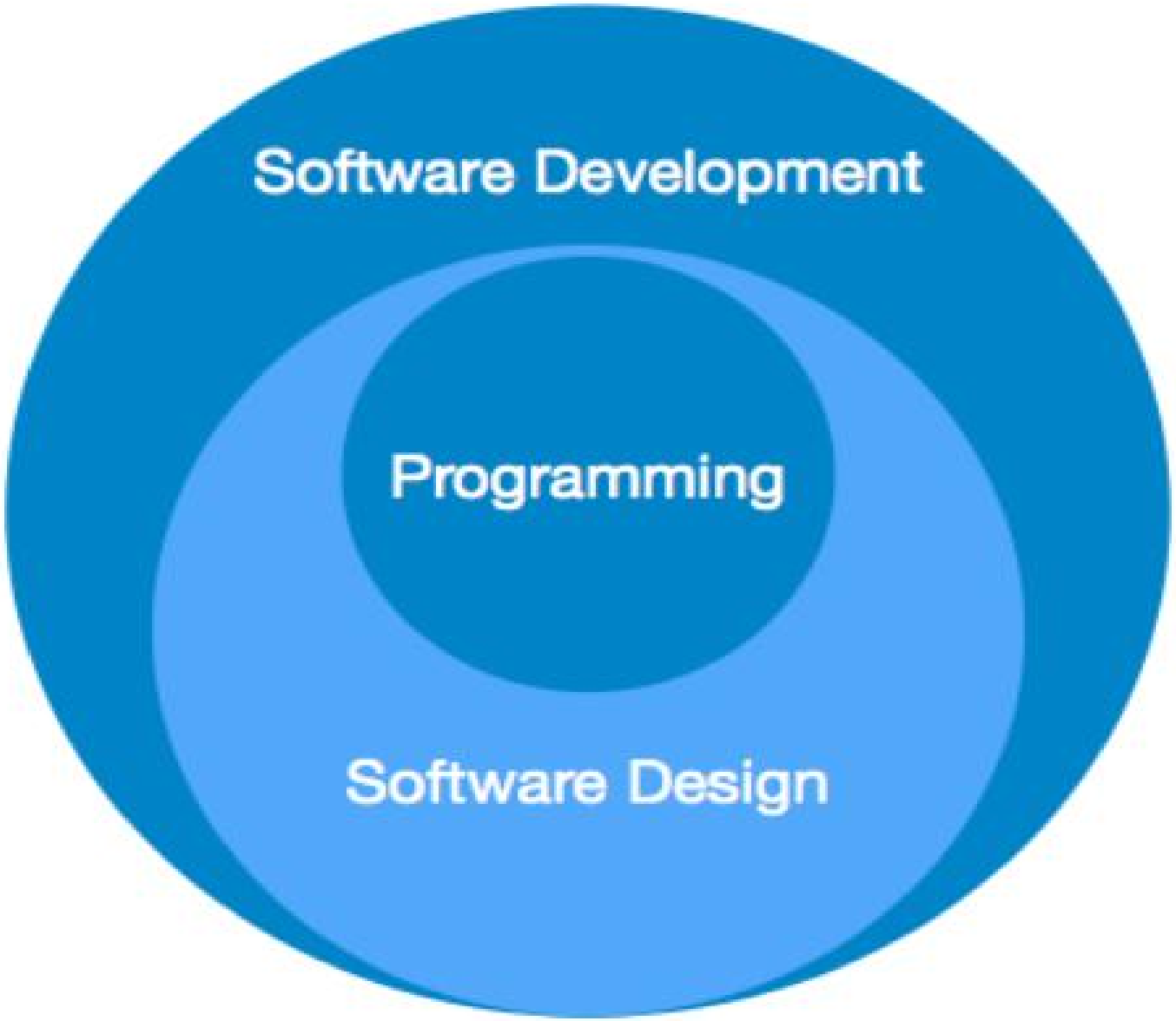
- **S-type (static-type)** - This is a software, which works strictly according to defined specifications and solutions. The solution and the method to achieve it, both are immediately understood **before coding.** The s-type software is least subjected to changes hence this is the simplest of all.

F**or example**, calculator program for mathematical computation.

- **P-type (practical-type)** - This is a software with a collection of procedures. This is defined by exactly what procedures can do. In this software, the specifications can be described but the solution is not obvious instantly.
- For example, gaming software.
- **E-type (embedded-type)** - This software works closely as the requirement of real-world environment. This software has a high degree of evolution as there are various changes in laws, taxes etc. in the real world situations. For example, Online trading software.

Software Paradigms

- Software paradigms refer to the methods and steps, which are taken while designing the software.
- There are many methods proposed and are in work today, but we need to see **where in the software engineering these paradigms stand**. These can be combined into various categories, though each of them is contained in one another:


Software Development
Programming
Software Design

Software Development Paradigm

- This Paradigm is known as software engineering paradigms where all the engineering concepts pertaining to the development of software are applied.
- It includes various researches and requirement gathering which helps the software product to build. It consists of
- Requirement gathering
- Software design
- Programming

Software Design Paradigm

- This paradigm is a part of Software Development and includes:
- Design
- Maintenance
- Programming

Programming Paradigm

- This paradigm is related closely to programming aspect of software development. This includes:

-Coding

-Testing

-Integration

Need of Software Engineering

- The need of software engineering arises because of higher rate of change in user requirements and environment on which the software is working.
- **Large software** - It is easier to build a wall than to a house or building, likewise, as the size of software become large engineering has to step to give it a scientific process.
- **Scalability-** If the software process were not based on scientific and engineering concepts, it would be easier to re-create new software than to scale an existing one.
- **Cost-** As hardware industry has shown its skills and huge manufacturing has lower down he price of computer and electronic hardware. But the cost of software remains high if proper process is not adapted.
- **Dynamic Nature-** The always growing and adapting nature of software hugely depends upon the environment in which user works. If the nature of software is always changing, new enhancements need to be done in the existing one. This is where software engineering plays a good role.
- **lity Management-** Better process of software development provides better and quality software product.

Characteristics of good software

- A software product can be judged by what it offers and how well it can be used. This software must satisfy on the following grounds:
- Operational
- Transitional
- Maintenance

Operational

- This tells us how well software works in operations. It can be measured on:
- Budget
- Usability
- Efficiency
- Correctness
- Functionality
- Dependability
- Security
- Safety

Transitional

- This aspect is important when the software is moved from one platform to another:
- Portability
- Interoperability
- Reusability
- Adaptability

Maintenance

- This aspect briefs about how well a software has the capabilities to maintain itself in the ever-changing environment:
- Modularity
- Maintainability
- Flexibility
- Scalability
- In short, Software engineering is a branch of computer science, which uses well-defined engineering concepts required to produce efficient, durable, scalable, in-budget and on-time software products.

Review Questions

1. What is software engineering?
2. Explain the characteristics of a good sofware
3. Discuss in detail on Software Paradigms
4. When you know programming, what is the need to learn software engineering concepts?

Software Development Life Cycle

- Software Development Life Cycle, SDLC for short, is a well-defined, structured sequence of stages in software engineering to develop the intended software product.

SDLC
Communication
Requirement Gathering
Feasibility Study
System Analysis
Software Design
Coding
Testing
Integration
Implementation
Operations & Maintenance
Disposition

Communication

- This is the first step where the user initiates the request for a desired software product.
- He contacts the service provider and tries to negotiate the terms.
- He submits his request to the service providing organization in writing.

Requirement Gathering

- This step onwards the software development team works to carry on the project.
- The team holds discussions with various stakeholders from problem domain and tries to bring out as much information as possible on their requirements.
- The requirements are contemplated and segregated into user requirements, system requirements and functional requirements. The requirements are collected using a number of practices as given -

- studying the existing or obsolete system and software,
- conducting interviews of users and developers,
- referring to the database or
- collecting answers from the questionnaires.

Feasibility Study

- After requirement gathering, the team comes up with a rough plan of software process.
- At this step the team analyzes if a software can be made to fulfill all requirements of the user and if there is any possibility of software being no more useful.
- It is found out, if the project is financially, practically and technologically feasible for the organization to take up.
- There are many algorithms available, which help the developers to conclude the feasibility of a software project.

System Analysis

- At this step the developers decide a roadmap of their plan and try to bring up the best software model suitable for the project.
- System analysis includes Understanding of software **product limitations,** learning system related problems or changes to be done in existing systems beforehand, identifying and addressing the impact of project on organization and personnel etc.
- The project team analyzes the **scope** of the project and plans the schedule and resources accordingly.

Software Design

- Next step is to bring down whole knowledge of requirements and analysis on the desk and design the software product.
- The inputs from users and information gathered in requirement gathering phase are the inputs of this step.
- The output of this step comes in the form of two designs; **logical design and physical design**.
- Engineers produce meta-data and data dictionaries, logical diagrams, data-flow diagrams and in some cases pseudo codes.

Coding

- This step is also known as programming phase.
- The implementation of software design starts in terms of writing program code in the suitable programming language and developing error-free executable programs efficiently.

Testing

- Software testing is done while coding by the developers and thorough testing is conducted by testing experts at various levels of code such as **module testing, program testing, product testing, in-house testing and testing the product at users end**.
- Early discovery of errors and their remedy is the key to reliable software.

Integration

- Software may need to be integrated with the libraries, databases and other program(s).
- This stage of SDLC is involved in the integration of software with outer world entities.

Implementation

- This means installing the software on user machines.
- At times, software needs post-installation configurations at user end.
- Software is tested for portability and adaptability and integration related issues are solved during implementation.

Operation and Maintenance

- This phase confirms the software operation in terms of more efficiency and less errors.
- If required, the users are trained on, or aided with the documentation on how to operate the software and how to keep the software operational.
- The software is maintained timely by updating the code according to the changes taking place in user end environment or technology.
- This phase may face challenges from hidden bugs and real-world

Disposition

- As time elapses, the software may decline on the performance front. It may go completely obsolete or may need intense upgradation. Hence a pressing need to eliminate a major portion of the system arises.
- This phase includes archiving data and required software components, closing down the system, planning disposition activity and terminating system at appropriate end-of-system time.

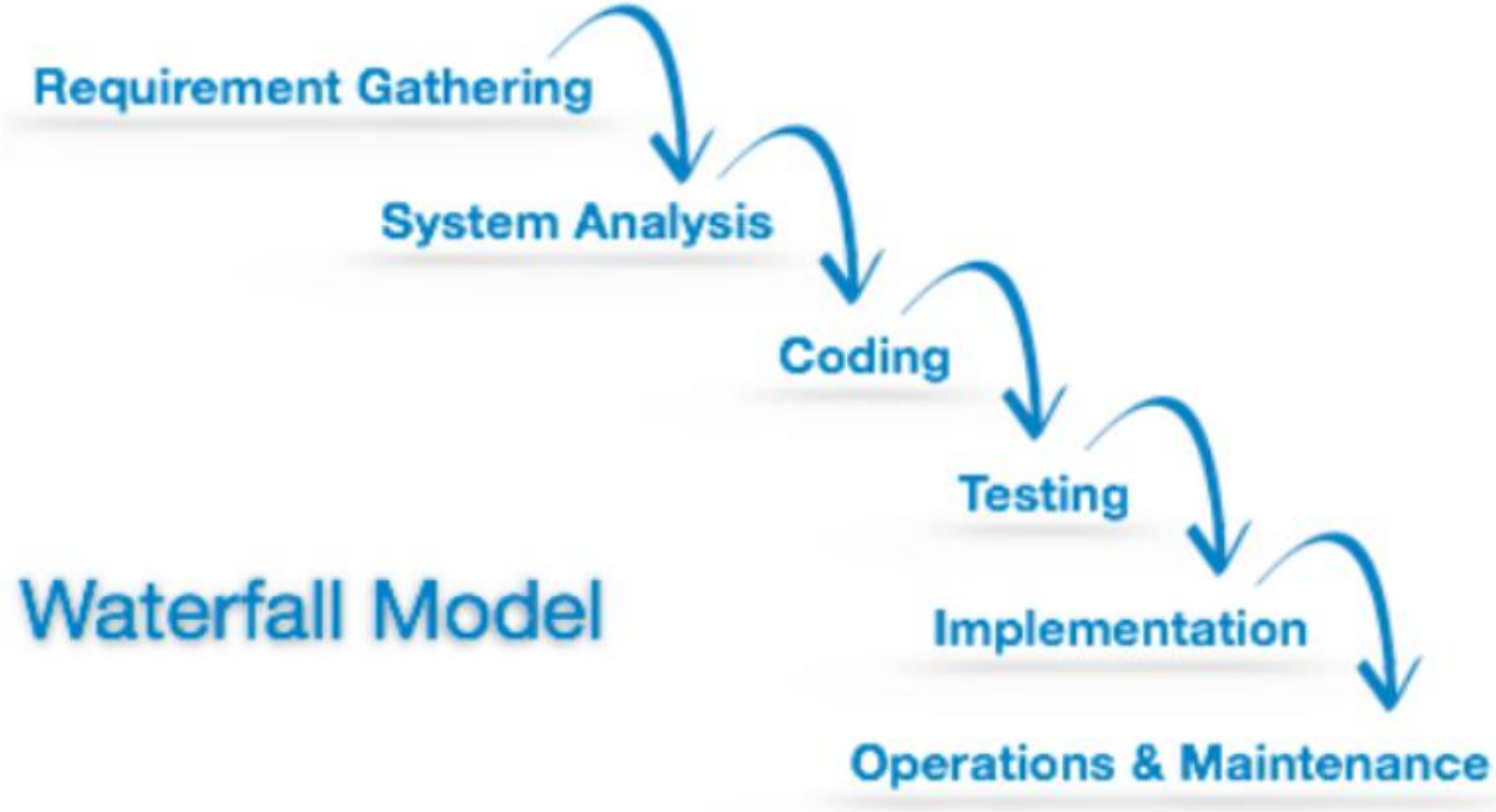
Software Development Paradigm

- The software development paradigm helps developer to select a **strategy** to develop the software.
- A software development paradigm has its own set of tools, methods and procedures, which are expressed clearly and defines software development life cycle.
- A few of software development paradigms or process models are defined as follows:

Waterfall Model

- Waterfall model is the simplest model of software development paradigm.
- It says the all the phases of SDLC will function one after another in linear manner.
- That is, when the first phase is finished then only the second phase will start and so on.

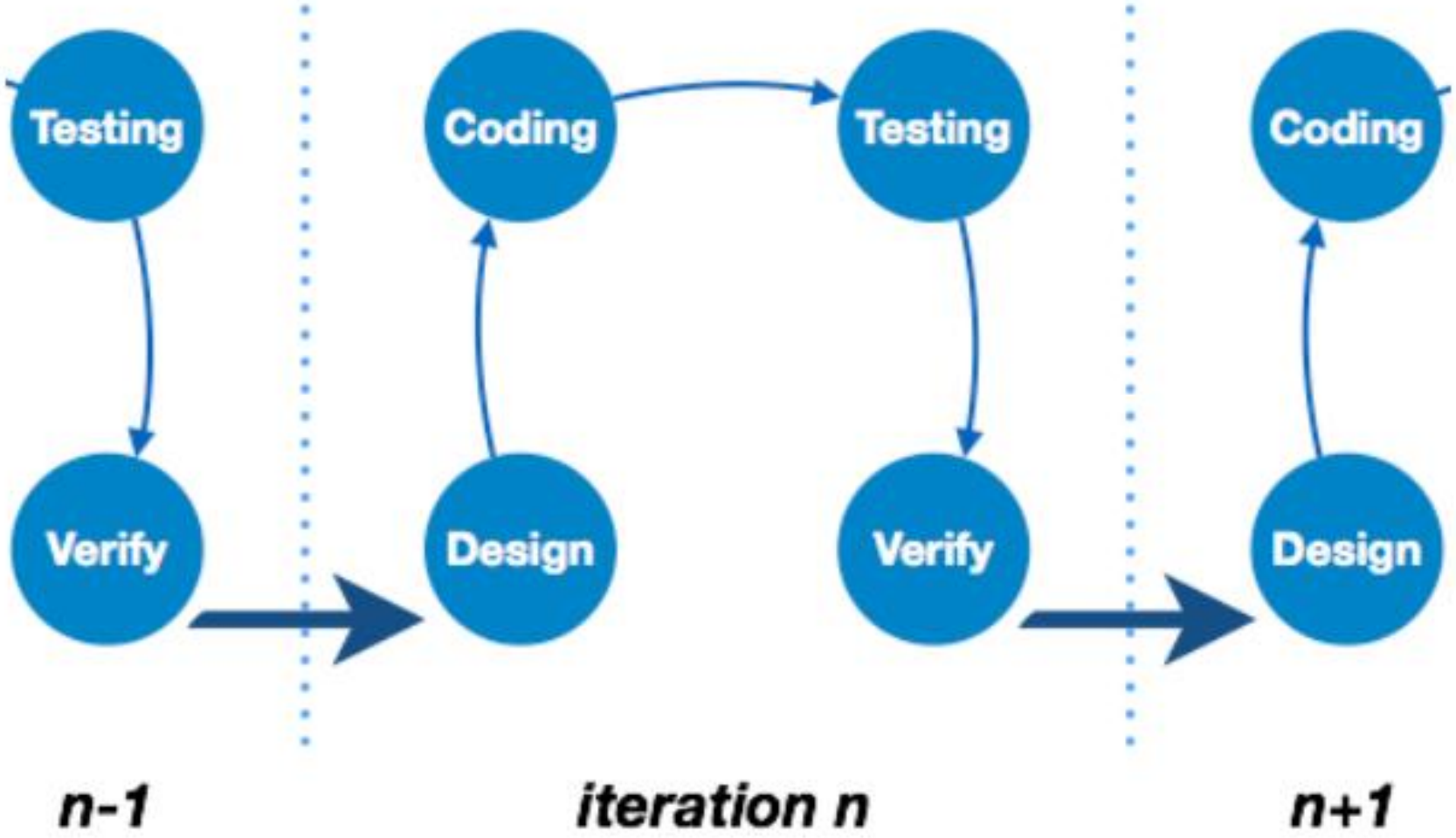
Waterfall Model



- This model assumes that everything is carried out and taken place perfectly as planned in the previous stage and there is no need to think about the past issues that may arise in the next phase.
- This model does not work smoothly if there are some issues left at the previous step.
- The sequential nature of model does not allow us go back and undo or redo our actions.

Iterative Model

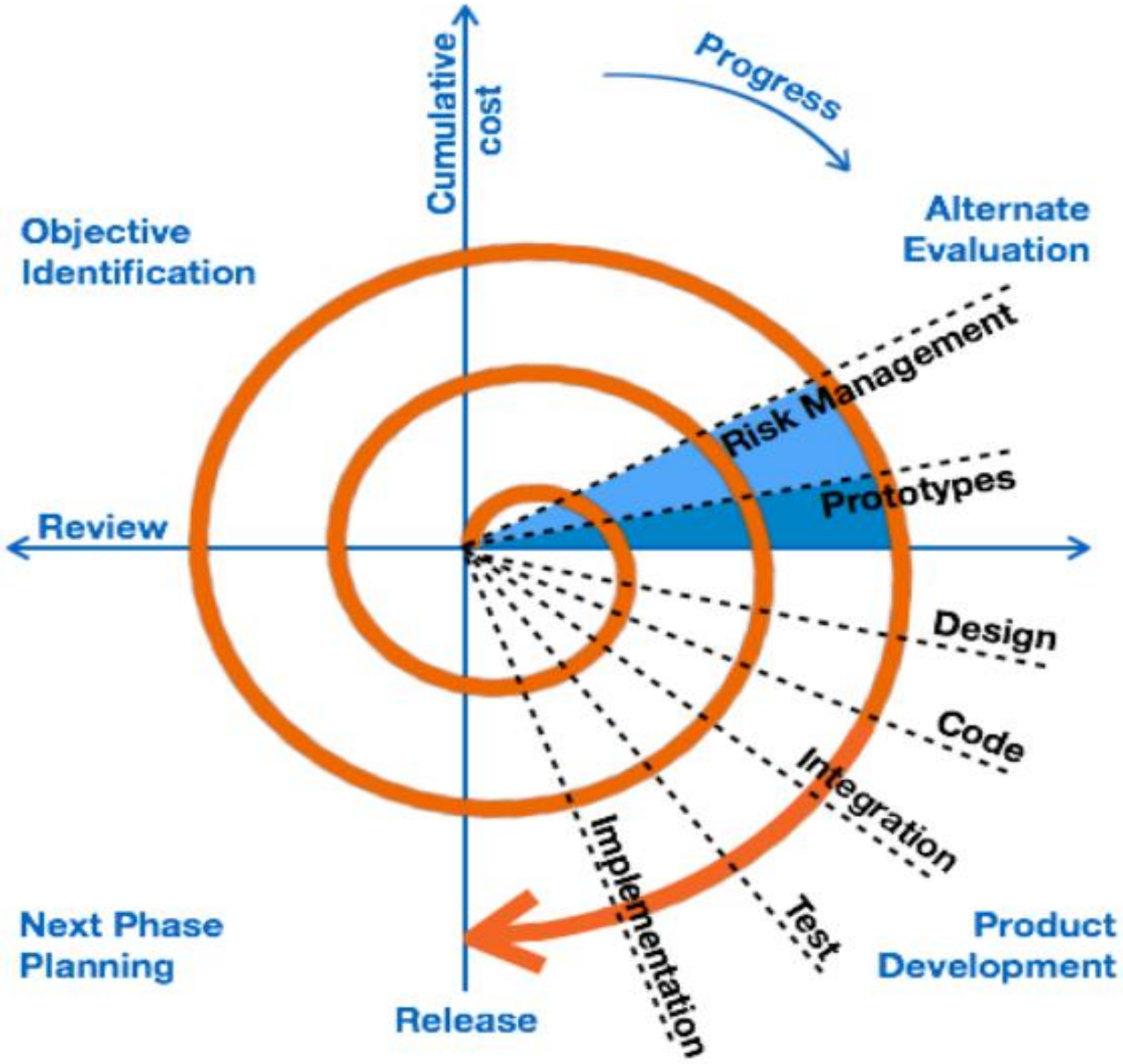
- This model leads the software development process in iterations.
- It projects the process of development in cyclic manner repeating every step after every cycle of SDLC process.


Testing
Verify
Coding
Design
Testing
Verify
Coding
Design
n-1
iteration n
n+1

- The software is first developed on very small scale and all the steps are followed which are taken into consideration. Then, on every next iteration, more features and modules are designed, coded, tested and added to the software.
- Every cycle produces a software, which is complete in itself and has more features and capabilities than that of the previous one.
- After each iteration, the management team can do work on risk management and prepare for the next iteration.
- Because a cycle includes small portion of whole software process, it is easier to manage the development process but it consumes more resources.

Spiral Model

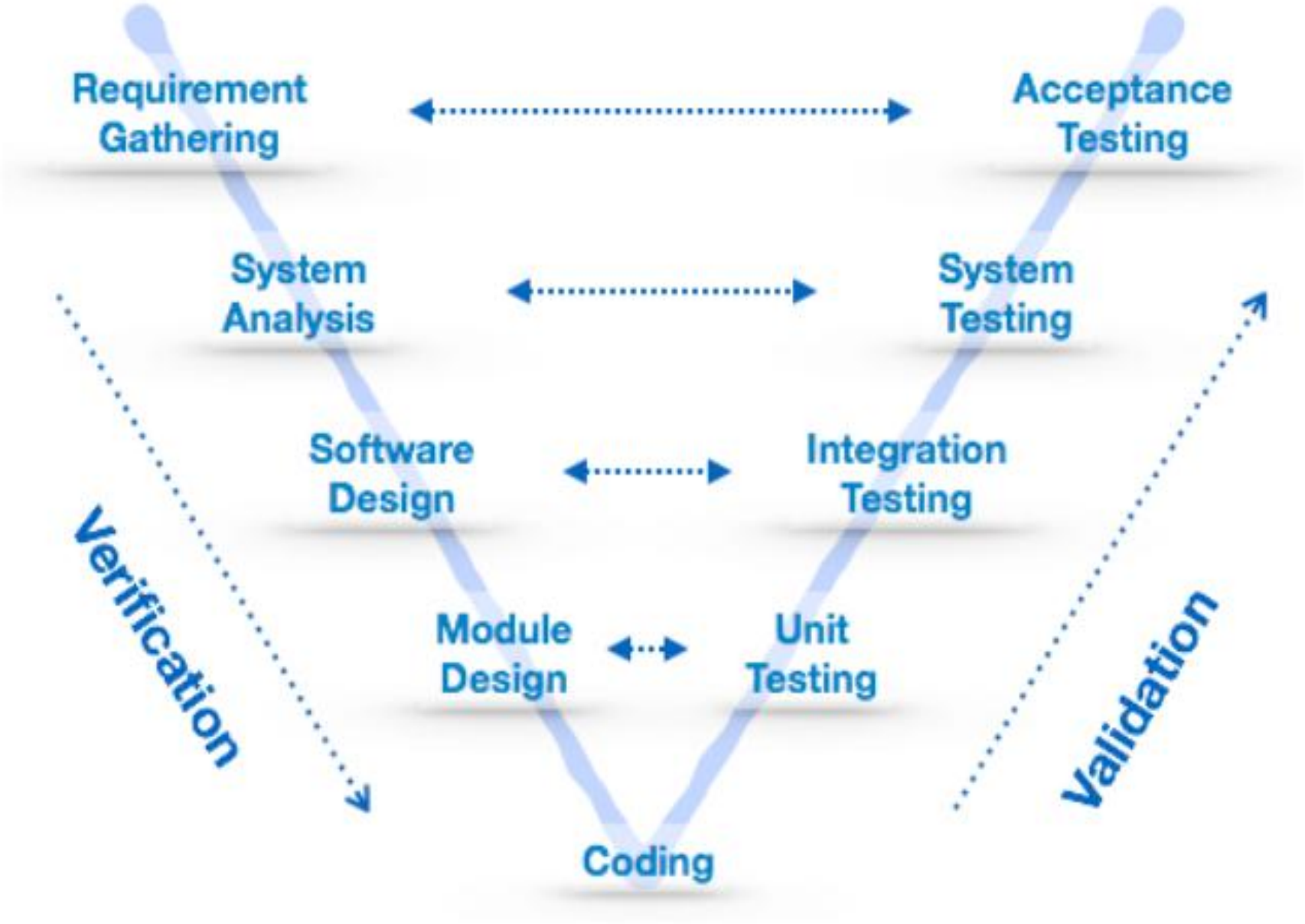
- Spiral model is a combination of both, iterative model and one of the SDLC model.
- It can be seen as if you choose one SDLC model and combine it with cyclic process (iterative model).


Cumulative cost
Progress
Objective Identification
Alternate Evaluation
Risk Management
Prototypes
Review
Design
Code
Integration
Implementation
Test
Next Phase Planning
Product Development
Release

- This model considers risk, which often goes un-noticed by most other models. The model starts with determining objectives and constraints of the software at the start of one iteration. Next phase is of prototyping the software.
- This includes risk analysis. Then one standard SDLC model is used to build the software.
- In the fourth phase of the plan of next iteration is prepared.

V model

- The major drawback of waterfall model is we move to the next stage only when the previous one is finished and there was no chance to go back if something is found wrong in later stages.
- V-Model provides means of testing of software at each stage in reverse manner.


Requirement Gathering
Acceptance Testing
System Analysis
System Testing
Software Design
Integration Testing
Module Design
Unit Testing
Coding
Verification
Validation

- At every stage, test plans and test cases are created to verify and validate the product according to the requirement of that stage.
- For example, in requirement gathering stage the test team prepares all the test cases in correspondence to the requirements.
- Later, when the product is developed and is ready for testing, test cases of this stage verify the software against its validity towards requirements at this stage.
- This makes both verification and validation go in parallel.
- This model is also known as **verification and validation** model.

Big Bang Model

- This model is the simplest model in its form.
- It requires little planning, lots of programming and lots of funds. This model is conceptualized around the big bang of universe.
- As scientists say that after big bang lots of galaxies, planets and stars evolved just as an event. Likewise, if we put together lots of programming and funds, you may achieve the best software product.

Time
Efforts
Resources
Big Bang
Software

- For this model, very small amount of planning is required. It does not follow any process, or at times the customer is not sure about the requirements and future needs. So the input requirements are arbitrary.
- This model is not suitable for large software projects but good one for learning and experimenting.

Review Questions

1. What is software process or Software Development Life Cycle (SDLC)?
2. What are SDLC models available?
3. What are various phases of SDLC?
4. Which SDLC model is the best?